Universidad Tecnológica de Bolivar
Ingeniería de Software Código 1206

Parcial práctico de Ingeniería de Software

La última transmisión

Integrantes

Zulianys Liseth Orozco Chavez	T00078330
Juan David Hernandez Avila	T00081559

Cartagena, octubre de 2026

1. Fundamentos: problema, alcance y calidad

Un grupo de escritores necesita crear y modificar escenas de una ficción interactiva sin depender de programadores; el motor propuesto carga narraciones desde archivos, valida su contenido y conserva el estado de la partida para evitar cierres inesperados. Los actores clave son: la escritora (editar escenas sin código), el diseñador (decisiones con consecuencias), la coordinadora (detectar y corregir errores), el lector (jugar y reiniciar) y el equipo (entregar motor y pruebas). El alcance incluye carga de historias, estado inicial, condiciones y efectos booleanos, validación, finales y reinicio, con ejecución local y dos historias de referencia; excluye cuentas, base de datos, guardado persistente, internet, editor visual y multimedia; queda pendiente decidir sobre el retroceso en decisiones. Los riesgos principales son contenido inválido (mitigado con validación previa) y acoplamiento del motor a una historia (mitigado con evaluación desde datos y prueba con otra historia). Se priorizan modificabilidad del contenido (RNF01), claridad de errores (RNF02) y confiabilidad (RNF03), sin dar relevancia a rendimiento, seguridad ni portabilidad avanzada. Finalmente, se asegura trazabilidad de autoría, instalación reproducible y licencias claras; el motor no guarda datos personales y requiere disciplina en el registro de dependencias.

2. Proceso de desarrollo y alternativa considerada

El proyecto *La última transmisión* se desarrollará de forma incremental e iterativo, construyendo el sistema por partes y verificando cada una antes de integrar la siguiente. Los incrementos principales son: carga de historias desde archivos JSON, validación de estructura, ejecución de escenas y opciones, gestión del estado y efectos, finalización y reinicio, e integración con pruebas. Este enfoque permite detectar defectos temprano y ajustar el diseño progresivamente. Como alternativa se consideró el modelo en cascada, útil con requisitos estables pero menos flexible ante errores. Se eligió el enfoque incremental porque permite verificar funciones críticas desde etapas tempranas, corregir defectos antes de que afecten al sistema completo y asegurar requisitos clave. El equipo dividirá tareas, coordinará frecuentemente y validará cada funcionalidad con pruebas y evidencias.

3. Requerimientos

3.1. Funcionales

- **RF1.1 Carga de historia externa:** El sistema carga datos desde archivos JSON independientes.
- **RF1.2 Inicialización del estado:** Se establece escena inicial y variables booleanas según el archivo.
- **RF2.1 Presentación de la escena:** Se muestra en pantalla el texto de la escena activa.
- **RF2.2 Filtrado de opciones:** Solo se presentan opciones habilitadas según condiciones.
- **RF3.1 Transición de escenas:** Una elección válida lleva al destino indicado.
- **RF3.2 Inmunidad ante errores:** Entradas inválidas se rechazan sin alterar el estado.
- **RF4.1 Aplicación de efectos:** Las elecciones actualizan variables en memoria hasta el fin de la partida.
- **RF5.1 Detección de final:** El sistema reconoce escenas finales y muestra desenlace.
- **RF5.2 Reinicio de sesión:** Tras final o bloqueo, se restaura el estado inicial.
- **RF5.3 Detección de bloqueo:** Si ninguna opción está habilitada, se informa bloqueo y se ofrece reinicio.
- **RF6.1 Validación previa:** Antes de ejecutar, se rechazan archivos con errores críticos.
- **RF6.2 Diagnóstico de validación:** Al rechazar un archivo, se muestra mensaje claro con causa e ID afectado.

3.2. No funcionales

- **RNF01 Modificabilidad:** Una segunda historia debe ejecutarse sin modificar el motor.
- **RNF02 Claridad de errores:** Cada error produce diagnóstico identificable y evita partidas inválidas.
- **RNF03 Confiabilidad:** Todas las pruebas funcionales e integración deben completarse correctamente.

3.3. Preguntas y supuestos

Durante el parcial no hubo respuesta del cliente, así que cada pregunta se resolvió con un supuesto: P1: retroceso en decisiones → versión inicial no lo implementa, solo se reinicia la partida. P2: variables iniciales → deben ser booleanas, valores no válidos se rechazan como error. P3: orden de opciones → se muestran según el archivo y el lector elige por número; el motor valida por ID.

3.4. Defecto detectado en requisitos

- **Problema:** RF5.1 mezclaba escenas finales con bloqueadas, ocultando errores.
- **Corrección:** RF5.1 limitado a finales; se crea RF5.3 para bloqueos con criterio propio; RF5.2 conserva su ID.
- **Artefactos afectados:** modelo de estados distingue *Finalizada* y *Bloqueada*; pruebas incluyen caso de bloqueo.
- **Detección:** revisión de requisitos con apoyo de IA.
- **Otra corrección:** RF3.2 unificado en "rechazar" entradas inválidas (antes decía "ignorar").

4. Decisiones de arquitectura

Para el desarrollo de *La última transmisión* se tomaron decisiones que organizan el sistema, facilitan mantenimiento y aseguran la correcta ejecución de historias interactivas.

4.1. Decisión 1: Separación contenido y lógica

Se eligió almacenar historias en archivos JSON externos y mantener la lógica en el motor. Esto permite modificar narraciones sin tocar el código, aunque requiere estructura consistente y validación previa.

4.2. Decisión 2: Separación de responsabilidades

El sistema se divide en componentes: presentación, administrador de sesión, motor de historias, cargador/validador y modelos de dominio. Esto mejora comprensión y pruebas, aunque exige definir comunicación clara entre módulos.

4.3. Decisión 3: Validación previa

Las historias se validan antes de ejecutarse. Así se evitan fallos durante la partida, a costa de mantener reglas de validación claras y diagnósticos específicos.

4.4. Decisión 4: Control del estado

Las opciones se verifican antes de modificar variables. El sistema distingue finales y bloqueos, y el reinicio restaura estado inicial. Esto asegura consistencia y facilita pruebas de transiciones y efectos.

4.5. Relación con atributos de calidad

Las decisiones refuerzan mantenibilidad, fiabilidad y facilidad de prueba: separar contenido y lógica facilita cambios; dividir responsabilidades mejora corrección; validar evita errores; y controlar selecciones protege consistencia. En conjunto, establecen una base arquitectónica sólida para ampliar y mantener el sistema.

5. Descripción General de la Versión 1.0

La **Versión 1.0** representa la primera entrega funcional del motor de historias interactivas por consola. El sistema permite desacoplar totalmente el contenido narrativo de la lógica del programa, permitiendo a escritores editar y extender relatos mediante archivos JSON sin tocar el código Python.

5.1. Características implementadas

- Carga e interpretación de archivos JSON externos estructurados.
- Validación estricta previa del contenido narrativo (detección de 9 categorías de error con diagnósticos detallados).
- Gestión del ciclo de vida y estado mutable de la sesión de juego en memoria.
- Evaluación de condiciones lógicas booleanas para habilitar o deshabilitar opciones dinámicamente.
- Aplicación de efectos que actualizan variables del relato tras tomar decisiones.
- Detección automática de desenlaces (escenas finales) y situaciones de bloqueo.
- Reinicio limpio de la partida restaurando el estado original.

6. Arquitectura y Módulos del Sistema

El proyecto sigue una arquitectura modular en capas con separación clara de responsabilidades:

Módulo	Ruta	Responsabilidad	Requisitos asociados
Dominio	`motor/dominio.py`	Modelado de entidades mediante `dataclasses` (`Historia`, `Escena`, `Opcion`, `EstadoSesion`).	Estructura base
Cargador	`motor/cargador.py`	Lectura de archivos JSON, invocación de validaciones y construcción del dominio.	RF1.1, RF1.2
Validador	`motor/validador.py`	Inspección estática del contenido antes de iniciar; verifica las 9 reglas de integridad.	RF6.1, RF6.2
Sesión	`motor/sesion.py`	Motor de juego: evalúa condiciones, aplica efectos, gestiona transiciones, reinicios y bloqueos.	RF2.2, RF3.1, RF3.2, RF4.1, RF5.1, RF5.2, RF5.3
Interfaz	`motor/interfaz.py`	Interacción por consola (CLI): renderizado de escenas, captura de opciones y control de flujo.	RF2.1, RF3.2
Diagnóstico	`motor/diagnostico.py`	Definición de excepciones y diagnósticos estructurados (`ErrorValidacion`, `Diagnostico`).	RF6.2
Punto de entrada	`main.py`	Orquestador principal de ejecución por línea de comandos.	CLI

7. Verificación de Requerimientos No Funcionales (RNF Medibles)

Los requerimientos no funcionales se definieron como medibles y verificables mediante pruebas objetivas y automatizadas:

ID	Atributo	Requisito medible	Método de verificación	Resultado obtenido
RNF01	Modificabilidad	Una segunda historia con al menos 3 escenas, 1 condición y 1 efecto debe ejecutarse sin modificar el código del motor.	Ejecución de `el_laboratorio_olvidado` en partida interactiva y en la prueba de integración `test_segunda_historia`.	**Cumple (100 %):** la historia alternativa se completó sin alterar ningún archivo de `motor/`.
RNF02	Claridad de errores	Cada una de las 9 categorías de error de contenido debe producir un diagnóstico identificable con campo o ID, impidiendo abrir una partida inválida.	Clase de pruebas parametrizadas `TestU4Validacion` (10 casos negativos).	**Cumple (100 %):** todos los casos generaron una excepción `ErrorValidacion` con el campo o ID erróneo.
RNF03	Confiabilidad	Las 6 pruebas base exigidas (U1 a U5 e I1) y la suite completa deben pasar sin fallos ni cierres inesperados.	Ejecución completa de la suite con `pytest`.	**Cumple (100 %):** 30 pruebas ejecutadas, 30 aprobadas (0 fallos, 0 errores) en 0.15 s.

8. Versión 2 — Solicitud de cambio de la hora 6

La versión 1 se conserva en el commit base `f69bdc7`. El anexo sustituye el acuerdo provisional de P1: el lector puede deshacer una elección con un único nivel de retroceso. P1 queda resuelta; no se ofrece rehacer ni retroceso múltiple.

8.1. Requisito nuevo y criterios de aceptación

N07. El lector necesita corregir su última decisión y revertir sus consecuencias sin perder el progreso anterior.

RF7.1 — Deshacer la última elección válida. El sistema debe restaurar escena, variables y banderas al estado inmediatamente anterior a la última elección válida. Solo existe un nivel de deshacer: al usarlo se consume el historial. Una nueva elección válida crea un nuevo punto de restauración. Entradas inválidas no alteran el estado ni reemplazan el historial.

RF7.2 — Disponibilidad del deshacer. Al iniciar sesión no debe existir historial. La acción está disponible desde los finales E4, E5 y E6 para regresar a E3. Al reiniciar, se limpia el historial junto con la restauración definida por RF5.2.

Criterios de aceptación:

1. Al comenzar, deshacer no está disponible ni modifica el estado.
2. Desde E2, elegir `investigar` lleva a E3 con `conoce_origen = true`; deshacer regresa a E2 con `conoce_origen = false`.
3. Tras deshacer, un segundo intento no cambia nada. Una nueva elección válida habilita un nuevo deshacer.
4. Una opción inexistente o deshabilitada se rechaza y conserva el punto previo.
5. Desde E4, E5 o E6 se puede deshacer la elección final y volver a E3 con los valores anteriores.
6. Reiniciar devuelve la sesión a E1 con variables iniciales y elimina el historial; deshacer después no produce cambios.

8.2. Análisis de impacto

- **Requisitos:** Se incorpora N07, RF7.1 y RF7.2; RF5.2 conserva su identificador y coordina la limpieza del historial.

- **Modelo de sesión:** `Sesion` guarda snapshot profundo del estado previo a una elección válida y lo descarta al deshacer o reiniciar.
- **Flujo de elección:** Validación antes de reemplazar snapshot; entradas inválidas preservan historial.
- **Interfaz:** Se añade comando `deshacer`, disponible también en finales E4–E6.
- **Compatibilidad:** No se requieren cambios en JSON ni historias.
- **Pruebas:** U6 cubre restauración y consumo; U5 verifica reinicio; I1 recorre historia y retorno desde finales.

8.3. Backlog de cambio

Tarea	Estado	Artefacto
Definir RF7.1 y RF7.2	Hecho	`documentacion/requerimientos.md`
Guardar/restaurar snapshot	Hecho	`motor/sesion.py`
Exponer comando retroceso	Hecho	`motor/interfaz.py`
Cubrir U6, U5 e I1	Hecho	`pruebas/test_motor.py`
Actualizar modelos y trazabilidad	Hecho	`documentacion/Anexo/diagramas.md`
Ejecutar suite y registrar evidencia	Limitado (sin pytest)	`pruebas/resultados_pruebas.txt`

8.4. Trazabilidad V2

Cambio	Diseño/código	Verificación
RF7.1: restauración exacta	Snapshot en `Sesion`; diagrama de estados	U6: restauración y consumo
RF7.1: entrada inválida no reemplaza historial	Validación en `Sesion.elegir`	U6: opción inexistente/deshabilitada conserva snapshot
RF7.2: iniciar y reiniciar sin historial	Constructor y `reiniciar`	U5: reinicio limpia historial
RF7.2: deshacer desde finales	Interfaz ofrece deshacer en estado final	I1: retorno desde E4–E6

8.5. Resultado de verificación y demostración

Se verificaron manualmente los flujos V2 en `pruebas/resultados_pruebas.txt`. Demostración: iniciar historia, elegir `revisar` y luego `investigar`; deshacer regresa a E2 con `conoce_origen = no`; segundo intento no cambia nada; continuar en E3 con opciones `activar_sin_verificar` y `cortar`. Desde E4–E6, deshacer retorna a E3. Al reiniciar, deshacer no está disponible.

Todas las imágenes con las validaciones y funcionalidades probadas se encuentran en el documento de Anexo.

Link Anexo: Anexo